MATLAB-Based Design and Verilog HDL Verification of a 32-Tap FIR Filter

Nguyen Dang Khoa[1], Tran Gia Thanh[1], Chau Nhat Nguyen[1], Vo Hoang Tri[1], and Hoang Kien Quoc[1]

[1]Department of Semiconductor Integrated Circuits, FPT University, Ho Chi Minh City, Vietnam {dangbin1982..., giathanhgo1..., chaunhatnguyen2006..., vohoangtri1912..., hoangkienquoc0311...}@gmail.com

***Abstract*–This study focuses on the design, simulation, and verification of a Finite Impulse Response (FIR) low-pass filter. The objective is to allow low-frequency signal components to pass through while attenuating unwanted high-frequency components. The filter was designed and analyzed using MATLAB and Simulink, then implemented in hardware using Verilog HDL and simulated in ModelSim. The accuracy of the design was evaluated by comparing the output data between MATLAB and ModelSim using the root-mean-square error (RMSE) and maximum absolute error metrics. This study clarifies the relationship between digital signal processing algorithm design and hardware implementation, assessing the impact of fixed-point arithmetic precision on filter performance.Keywords: FIR filter, Low-pass filter, MATLAB, Simulink, Verilog HDL, ModelSim, Hamming window, Q15.**

Keywords–FIR filter, Low-pass filter, MATLAB, Simulink, Verilog HDL, ModelSim, Hamming window, Q15.

1. Introduction

1.1. Background

Digital signal processing (DSP) plays a crucial role in modern electronic systems, particularly in audio processing, telecommunications, biomedical systems, and multimedia applications. In practice, signals can contain unwanted frequency components, noise, or disturbances that affect the quality of the original information. Therefore, digital filters are used to process sampled signals by controlling which frequency components are allowed to pass through the system [1,2,3].

Among various digital filter architectures, the Finite Impulse Response (FIR) filter is widely used due to its guaranteed stability and inherent linear-phase capability. Compared to analog filters, digital FIR filters offer greater flexibility in modifying filter characteristics through software-based coefficient adjustments. Hardware description languages such as Verilog HDL allow digital filtering algorithms to be translated into dedicated hardware structures, making FIR filters highly suitable for implementation on digital hardware platforms, including Field Programmable Gate Arrays (FPGAs).

In this project, a 32-tap low-pass FIR filter was developed using a software-assisted design and verification methodology. MATLAB and Simulink were utilized for coefficient calculation and system analysis, while Verilog HDL and ModelSim were employed for digital logic synthesis and hardware simulation.

1.2. The role of FIR filters in digital signal processing

FIR filters are widely utilized in digital signal processing due to their intrinsic stability, as their non-recursive structure prevents feedback loop instability. Furthermore, when designed with symmetric coefficients, FIR filters exhibit exact linear phase, ensuring that all frequency components experience equal time delay and preventing phase distortion. Their hardware implementation is straightforward, relying entirely on shift registers, multipliers, and adders. FIR filters are essential components in audio processing, telecommunications, image processing, biomedical systems (such as ECG and EEG signal conditioning), and custom FPGA/ASIC hardware implementations [1,2].

1.3. Project Objectives

Design a 32-tap low-pass FIR filter with a specified sampling frequency $F_s = 8KHz$ and cutoff frequency $F_c = 1KHz$ using the Hamming window method.

Generate a dual-tone test signal comprising 500 Hz and 2500 Hz sinusoidal components, verifying in MATLAB/Simulink that the filter preserves the 500 Hz signal while suppressing the 2500 Hz component.

Quantize the filter coefficients and input samples into fixed-point integer format (Q15 for coefficients) for Verilog hardware simulation.

Construct Verilog filter modules and a testbench environment, performing ModelSim simulations across 161 signal samples.

Evaluate hardware accuracy by comparing ModelSim simulation results with MATLAB reference outputs using error curves, RMSE, and maximum absolute error.

1.4. Scope of Implementation

Tools	Description / Purpose
MATLAB (Filter Design)	Design the filter, calculate the coefficients, and plot the frequency response.
Simulink	Block diagram simulation: 2 sine sources → summing → filter → scope
Verilog	Hardware description of the 32-tap FIR filter and verification testbench.
ModelSim	Executes the Verilog simulation, displays signal waveforms, and exports output data.

The scope of this project is limited to hardware design and functional simulation; synthesis and deployment onto a physical FPGA platform were not performed.

2. Theoretical Background

2.1. Concept and principles of FIR filters

FIR filters compute output samples by forming a weighted linear combination of present and past input samples. The system maintains a finite sequence of input samples within a delay line [1]. As each new sample arrives, the input sequence shifts by one sample period, and the multiply-accumulate operation is executed to produce the corresponding output sample.

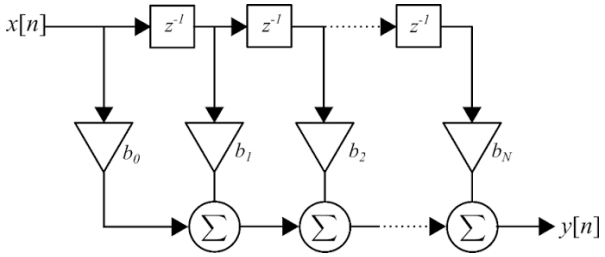


Figure 2.1 The concept of FIR design [2]

2.2. Mathematical equations of FIR

2.2.1 Main Equations

a) General equation of the FIR filter

$$y[n] = \sum_{k=0}^{N} b[k]x[n-k]$$

y[n]: output signal at sample n.
x[n-k]: current input sample or past sample.
b[k]: k-th filter coefficient
N: filter order

This is the core formula that can be used for all filters and is the formula used to implement the module via Verilog in Modelsim using multiplication and addition operations, and then comparing the results with those from Matlab [3].

b) Ideal impulse response $h_{\text{ideal}}[n]$

$$h_{ideal}[n] = \begin{cases} \frac{2F_c}{F_s} & n = \alpha \\ \frac{sin(2\pi\frac{F_c}{F_s}(n-\alpha))}{\pi(n-\alpha)}, & n \neq \alpha \end{cases}$$

This equation defines the impulse response of an ideal low-pass filter at the specified cutoff frequency, mapping the desired frequency response into discrete-time coefficients. Given the design parameters $F_s = 8kHz$, $F_c = 1kHz$, and filter length L = 32 taps, the symmetry center is located at $\alpha = \frac{L-1}{2} = 15.5$. Substituting these values into the equation yields the ideal impulse response sequence for n ∈ {0, ..., 31}. The resulting symmetric coefficient sequence enforces linear phase characteristics, forming the mathematical baseline for transforming ideal filter requirements into realizable tap weights.

c) Hamming Window

$$\omega[n] = 0.54 - 0.46\cos(\frac{2\pi n}{L-1})$$

d) The Hamming window truncates the infinitely long ideal impulse response to 32 discrete samples, yielding the realizable filter coefficients b[k]. Unlike a rectangular window, which introduces severe truncation discontinuities, the Hamming window smoothly tapers the coefficients near the boundaries. This significantly reduces passband and stopband ripple, improving stopband attenuation [4]. The trade-off for this improved attenuation is a slight widening of the transition band for a fixed number of taps. The windowed coefficients are subsequently utilized in Simulink and Verilog implementations.

e) Calculation of FIR filter coefficients b[k]

$$b[n] = h_{ideal}[n]\omega_{hamming}[n]$$

The coefficients b[k] dictate the system's filtering performance as they directly define the z-domain transfer function $H(z) = b[0] + b[1]\cdot z^{-1} + ... + b[N-1]\cdot z^{-(N-1)}$ and the continuous frequency response $H(and^{j\omega})$:

$$H(e^{j\omega}) = \sum_{k=0}^{N-1} b[k]e^{-j\omega k}$$

The overall frequency response $H(and^{j\omega})$ is formed by the complex sum of scaled exponentials weighted by b_k. Modifying the coefficient values b_k reshapes the magnitude and phase response curves. Consequently, the magnitude and arrangement of these tap weights determine which spectral components pass through and which undergo attenuation. In this design, the coefficients b[k] are configured to preserve a 500 Hz target signal while attenuating a 2500 Hz noise component. Reconfiguring this coefficient vector enables alternative filter responses, such as high-pass, band-pass, or band-stop transfer functions.

2.2.2. Auxiliary Equations

a) Sampling frequency and sampling period

$$F_s = \frac{1}{T_s},\ T_s = \frac{1}{F_s}$$

Fs: sampling frequency, in Hz.

Ts: sampling period, in seconds.

b) The sampling frequency Fs defines the rate at which continuous signals are discretized per second. In this project, Fs is set to 8 kHz, which corresponds to a sampling period Ts of 125 µs between consecutive samples. This parameter sets the fundamental baseline for filter synthesis, signal discretization, and time-domain analysis during system simulation.
c) Number of signal cycles within the observation window

$$N_{cycles} = fT_{obs}$$

f: signal frequency.

T_{obs}: observation time or simulation time.

N_{cycles}: the number of signal cycles within that time interval.

This relation enables verification of signal periodicity in MATLAB; for instance, setting an observation time of 0.02 s captures exactly 10 full cycles of a 500 Hz signal.

d) Nyquist frequency

$$F_N = \frac{F_s}{2}$$

e) The Nyquist frequency defines the upper boundary for alias-free signal representation. At a sampling rate of 8000 Hz, the Nyquist frequency is 4000 Hz (0 < Fc < Fs/2). This confirms that all test frequencies reside within the valid processing bandwidth of the system.
f) Group delay of linear-phase FIR filters

$$n_d = \frac{L-1}{2} = \frac{N}{2}$$

Calculating the group delay (15.5 samples, corresponding to approximately 1.94 ms) allows proper temporal alignment between input and output sequences during comparative analysis between MATLAB and ModelSim. Additionally, this parameter accounts for real-time latency and ensures temporal synchronization across digital hardware blocks.

g) Q15 coefficient quantization

$$B_{Q15}[n] = \text{round}\ (b[n]2^{15})$$

$b[n]$: initial real number coefficient.

$B_{Q15}[n]$: integer coefficient after quantization.

$\hat{b}[n]$: The real number coefficients are reconstructed from the Q15 value.

While MATLAB performs coefficient design using double-precision floating-point numbers, hardware implementations typically employ fixed-point integer arithmetic. To represent fractional numbers in the range [-1, 1) using 16-bit signed integers, the Q15 format is applied:

- 1 sign bit and 15 decimal bits.
- Quantization conversion: Q15 value = round(floating-point value × 2^{15}), which corresponds to scaling by 32768.
- Reconstruction conversion: Floating-point value = Q15 value / 32768.
- Dynamic range: from -1.0 to approximately +0.999969.

h) For example, an initial tap weight h[0] of -0.00064 yields -21 when scaled by 32768 and rounded. In ModelSim, multiplying two Q15 quantities expands the fractional width by 15 bits. Consequently, the accumulated sum must be arithmetically shifted right by 15 bits (divided by 2^{15}) to restore the original scale. Truncating lower-order bits during this shift operation introduces quantization error.
i) Error e[n], Maximum Absolute Error Emax, and RMSE

$$e[n] = y_{MATLAB[n]} - y_{ModelSim[n]}$$

$$E_{max} = \max_n |e[n]|$$

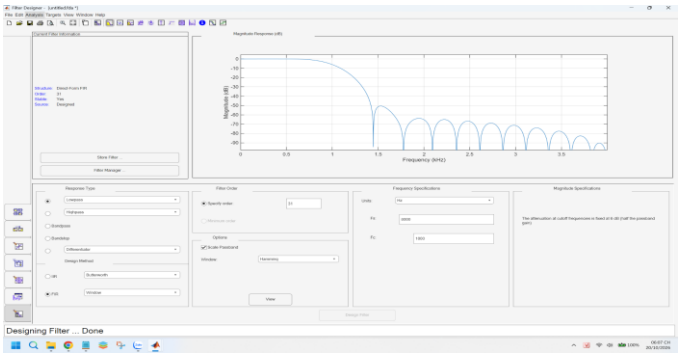
$$\text{RMSE} = \sqrt{\frac{1}{M}\sum_{n=0}^{M-1}(y_{MATLAB[n]} - y_{ModelSim[n]})^2}$$

Error metrics quantify the deviation between MATLAB double-precision reference out puts and fixed-point Verilog hardware outputs. The point error e[n] measures sample-by-sample differences, the maximum absolute error Emax identifies the peak deviation across the dataset, and the Root Mean Square Error (RMSE) evaluates overall statistical variance. These metrics facilitate validation of hardware behavior and highlight errors caused by coefficient quantization, bit truncation, or accumulator overflow. Prior to computing error metrics, delay compensation and scale matching must be applied to align the datasets.

3. FIR Filter Design and MATLAB Simulation

3.1. Design Parameters

The filter specifications were configured using the MATLAB Filter Designer tool. Instead of manually evaluating the ideal impulse response, the utility computes the 32-tap coefficient vector b[k] automatically. Design inputs included a sampling frequency $F_s = 8KHz$, a cutoff frequency $F_c = 1KHz$, and a filter order N = 31 (corresponding to L = 32 taps under $L = 32$). A low-pass response was selected using a Hamming window in direct form FIR structure, and the session parameters were exported as Filter Coefficient.fda.



[Figure 3.1 – Filter Designer GUI displaying design parameters and magnitude response]

3.2. Simulink Model and Simulation Results

The Simulink model FIR_Lowpass_Mode.slx comprises four fundamental processing blocks:

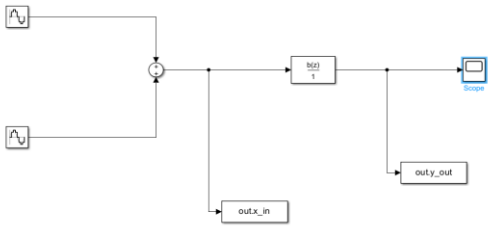
1. Two **Sine Wave** generator blocks, generating the 500 Hz passband tone and the 2500 Hz stopband tone.

2. A **Sum** block that combines the two sinusoidal components into a composite input signal.

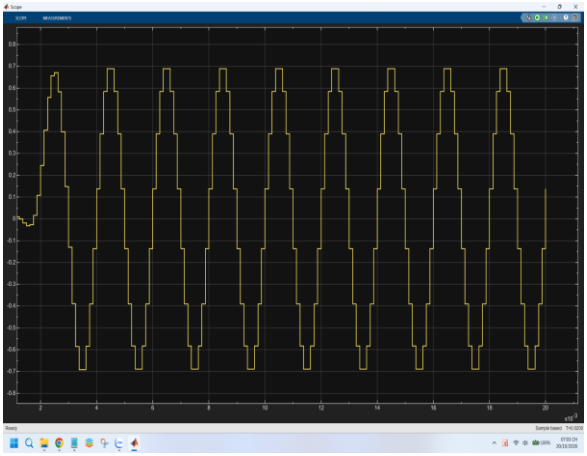
3. A **Discrete Filter** block parameterized with the 32 FIR tap weights.

4. A **Scope** block to visualize time-domain input and output waveforms.

The total simulation stop time was set to 0.02 s.



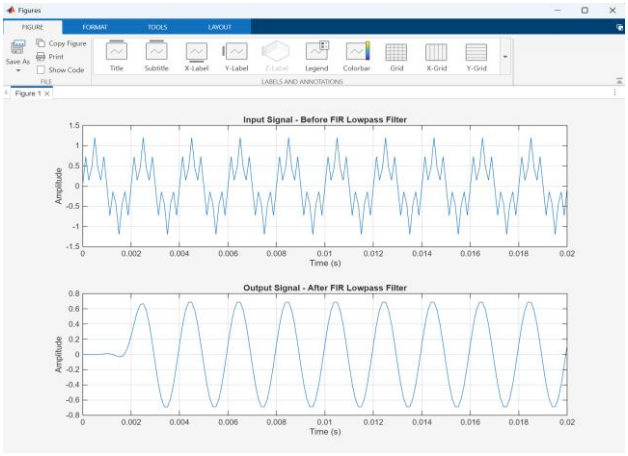
[Figure 3.2a – Simulink Block Diagram]



[Figure 3.2b – Output signal on Scope]

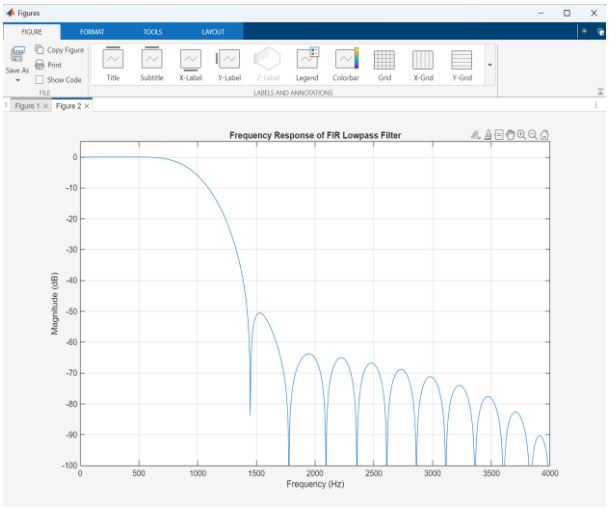
Results: The composite input signal, formed by summing a 500 Hz sine wave (amplitude 0.7) and a 2500 Hz sine wave (amplitude 0.5), performed as expected. The filtered output signal settled to a peak amplitude between -0.7 and +0.7, demonstrating complete suppression of the 2500 Hz component while preserving the 500 Hz fundamental tone. Without filtering, the peak input amplitude reached 0.7 + 0.5 = 1.2, exhibiting high-frequency distortion and ripple. Furthermore, the 0.02 s simulation window captures exactly 10 complete cycles of the fundamental waveform.

3.3. MATLAB Verification of FIR Filter Performance



[Figure 3.3a – Signal before and after filtering]

Filter performance was further evaluated via the MATLAB Command Window scripts. The unfiltered composite signal exhibits high-frequency ripple with peak values reaching ±1.2 due to phase addition of the 0.7 and 0.5 amplitude components. Following low-pass filtering, the output settles to a smooth 500 Hz sinusoid with an amplitude of 0.7, confirming complete attenuation of the 2500 Hz component. Additionally, initial latency ($t < 0.002s$) is visible during the transient phase, matching the theoretical filter group delay of 15.5 samples calculated previously.



[Figure 3.3b – Frequency response of a low-pass FIR filter]

Analysis: This plot illustrates the magnitude-frequency response of the 32-tap FIR filter synthesized using the Hamming window method.

Observations: The passband extends from 0 to $1000Hz$, preserving signal components without attenuation up to 1000 Hz where roll-off begins. The transition band spans from $1000Hz$ to approximately $1500Hz$, demonstrating steep frequency selectivity characteristic of a 32-tap design. Within the stopband (from $1500Hz$ upward), attenuation ranges between $-50dB$ and $-80dB$. This severe attenuation explains why the $2500Hz$ component is effectively eliminated from the output spectrum.

3.4. Exporting Data to Verilog

Following filter design in MATLAB, the floating-point tap weights were converted to fixed-point integer representations to ensure compatibility with Verilog hardware structures.

The coefficients were quantized into 16-bit signed Q15 format using the conversion formulation described in Section 2. The input signal samples were scaled by X_{SCALE} = 100 and rounded to 8-bit signed integers. The quantized vectors were exported as hex memory files (fir_input.hex and fir_coeff.hex) for loading into ModelSim. Reference datasets were saved in FIR_Lowpass_Reference.mat for hardware co-verification, ensuring format consistency between MATLAB and Verilog simulation environments.

4. Verilog Implementation and ModelSim Simulation

4.1. Architecture of the FIR Module

The fir32_lowpass module (defined in fir32_lowpass.v) implements the standard difference equation $y[n] = \Sigma\, b[k]\cdot x[n-k]$ across 32

taps. Key module signals include:

Execution steps per clock cycle:

1. **Sample Shifting:** The incoming sample x_{in} is loaded into the head register, while historical samples shift through the delay chain (z^{-1} elements).

2. **Multiply-Accumulate (MAC):** Each registered sample is multiplied by its corresponding Q15 coefficient, and the 32 products are summed into the acc register.

3. **Scaling & Truncation:** The contents of acc are arithmetically shifted right by 15 bits (dividing by 2^{15}) to rescale Q15 values back to the integer domain, updating y_{out}.

Hardware Design and Data Bit-Widths

The 32-tap FIR module is designed using synchronous sequential logic using an always @(posedge clk or negedge rst_n) block to manage clocked register updates and active-low asynchronous resets.

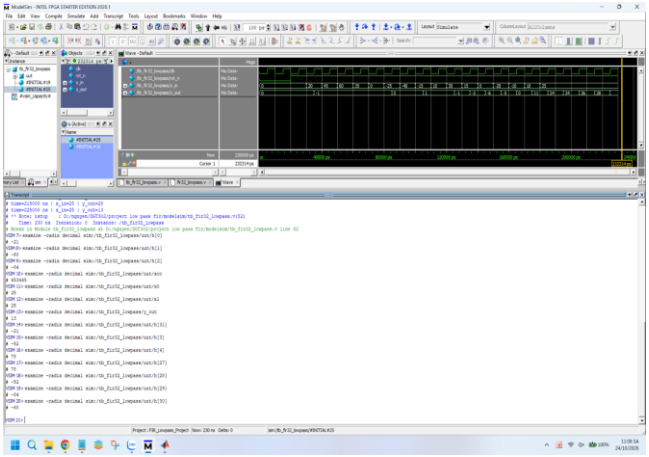
Inside the module, 32 input samples are registered using 8-bit signed array elements. The filter tap weights are stored in a 16-bit signed array h. To prevent register overflow during the accumulation of the 32 product terms, the accumulator variable acc is allocated a 40-bit bit-width (reg signed [39:0] acc).

Finally, the accumulated sum undergoes a 15-bit arithmetic right shift (acc >>> 15) to convert from Q15 fixed-point format back to integer scale before updating the output register y_{out}, which has a 24-bit bit-width.

Representative example from simulation: when acc = 453445, the output evaluates to y_{out} = 13. Verification: 453445 / 32768 ≈ 13.84, which truncates to **13** under 15-bit arithmetic right-shifting.

The coefficient values were also verified in ModelSim using the examine -radix decimal command, yielding: h[0] = −21, h[1] = −60, h[2] = −84, h[3] = −52, h[4] = 78, h[27] = 78, h[28] = −52, h[29] = −84, and h[30] = −60. All coefficient values **match** the quantized B_{Q15} vector generated in MATLAB.

Signal Name	Meaning
clk	System clock; processes one input sample per rising edge cycle.
rst_n	Active-low asynchronous reset; clears internal delay registers.
x_{in}	Input sample vector (8-bit signed integer).
y_{out}	Filtered output sample (signed integer).
h[0..31]	Array holding 32 signed 16-bit Q15 tap coefficients.
x_0, x_1, ...	Shift register array storing the 32 most recent input samples.
acc	Extended-width accumulator register to prevent arithmetic overflow.



[Figure 4.1 – Verifying coefficient values and accumulator registers in ModelSim]

4.2. Testbench Architecture

Because the FIR hardware module requires stimulus signals for operation, the testbench tb_fir32_lowpass.v serves as a simulation harness that executes the following tasks:

Generate system clock (clk) and asynchronous reset (rst_n) signals.

Read input stimulus vectors from external files and sequentially feed samples into x_{in}.

Instantiate the FIR Unit Under Test (UUT).

Log output samples y_{out} to a text file for post-simulation MATLAB comparison.

Terminate simulation via $stop once all input samples have been processed.

Testbench constructs are non-synthesizable and are used exclusively for verification within the simulation environment.

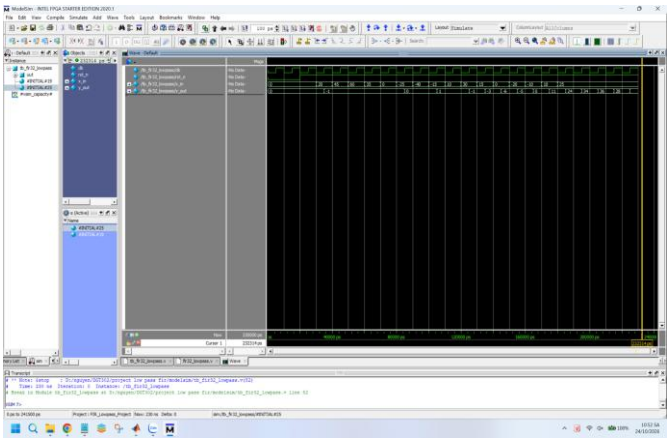
4.3. Reading Coefficients and Input Memory

Input Data Loading: The testbench executes the $readmemh system task to load fir_input.hex (generated in MATLAB) into the input_mem array. This memory array contains 161 input signal samples that are fed into the FIR module sequentially on each clock cycle.
Filter Coefficient Loading: The FIR module instantiates a 32-element array h[0:31] comprising 16-bit signed Q15 tap weights. The array is populated from fir_coeff.hex using $readmemh within an initial block during simulation initialization.
Sample Counter: Within the testbench, an integer index variable i tracks the number of processed samples, triggering simulation halt once all 161 samples complete execution.

The variable i serves as the sample counter within the testbench architecture. At simulation conclusion, i = 161 (see Figure 4.3b).

4.4. Simulation Execution over 161 Samples

Initial verification confirmed waveform behavior across early cycles. The input register x_{in} updates every clock cycle (0, 20, 45, 60, 35, 0, -25, -40, ...), while y_out initializes with low-amplitude transient responses (0, −1, 0, 1, ...) before building up amplitude, mirroring the transient latency observed in MATLAB.

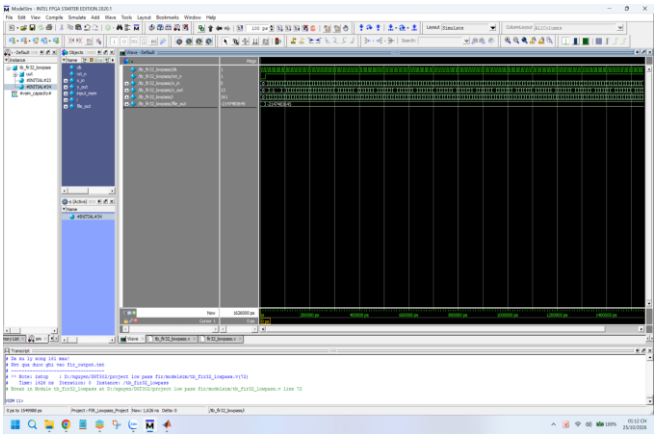


[Figure 4.3a – Initial ModelSim simulation waveforms during transient phase]

The testbench was subsequently executed across the full dataset of 161 samples. With a clock period of 10 ns, the simulation completed in **1626 ns**. The ModelSim Transcript window reported:

Successfully processed 161 samples!

Results written to fir_output.txt



[Figure 4.3b – Waveform and Transcript after running 161 samples]

4.5. Logging Output Data

The testbench initializes a file handle using $fopen and writes each calculated y_{out} sample using $fdisplay, generating an output file fir_output.txt containing exactly **161 records**.

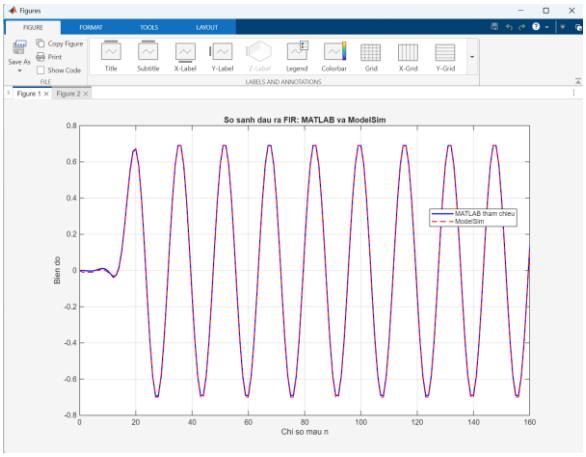
Representative output sequence:

Location	Value (on a scale of 100)
Initial 11 samples	0, −1, −1, −1, −1, −1, 0, 0, 1, 0, −1
Transient (sample 12 to 22)	−2, −4, −3, 1, 10, 24, 40, 55, 65, 67, 58
Steady-state sequence (repeating every 16 samples)	13, 39, 58, 69, 69, 58, 39, 13, −14, −40, −59, −70, −70, −59, −40, −14

During steady state, the output settles into a periodic pattern repeating every **16 samples**, which corresponds to a 500 Hz sinusoidal wave at Fs = 8000 Hz (8000 / 500 = 16 samples per period). The peak magnitude of approximately ±70 matches the 0.69 amplitude observed on the Simulink Scope when accounting for signal scaling.

5. Results and Discussion

5.1. Comparative Analysis of MATLAB and ModelSim Outputs



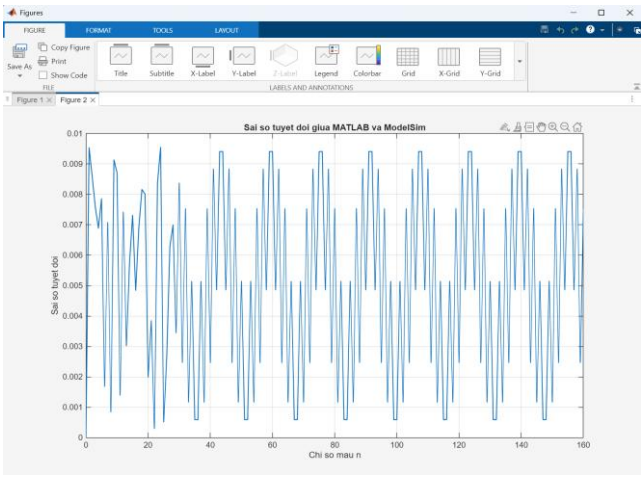
[Figure 5.1 – Comparison of MATLAB and ModelSim outputs]

Comparison of the output waveforms demonstrates that the MATLAB reference signal (solid blue line) and ModelSim fixed-point output (dashed red line) overlap closely across all 161 samples. The peaks, zero-crossings, and overall oscillatory shapes exhibit high consistency, confirming that the Verilog implementation accurately mirrors the theoretical DSP model. Minor deviations are attributable to fixed-point quantization and bit truncation inherent in hardware arithmetic operations.

5.2. Evaluation of Error Metrics and Precision

Summary of calculated error metrics exported from MATLAB:

Metric	Value
Input sample count	161
ModelSim output sample count	161
Maximum absolute error (Emax)	**0.00954506** (≈ 0.0095)
Root Mean Square Error (RMSE)	**0.00601066** (≈ 0.0060)



[Figure 5.3 – RMSE and maximum absolute error reported in the MATLAB Command Window]

The sample-by-sample error curve e[n] confirms that discrepancies fluctuate within a tightly bounded range without systematic divergence. Most sample errors remain minimal, indicating strong alignment between ModelSim hardware simulation and MATLAB reference calculations. Residual errors stem primarily from fixed-point numeric constraints, including Q15 coefficient quantization, input quantization, and truncation during accumulator shift operations.

A peak absolute error Emax of 0.00954506 represents the single largest discrepancy observed across all 161 samples. Meanwhile, the RMSE value of 0.00601066 reflects the overall statistical variance across the entire dataset. The low RMSE value confirms high fidelity between the fixed-point Verilog hardware implementation and the double-precision software model.

5.3. Comparison with Related Literature

To contextualize the performance and validity of this work, the 32-tap FIR filter design was compared against existing literature in hardware-based digital signal processing. Comparison criteria included tap length, hardware description methodology, numeric format, verification techniques, and arithmetic accuracy.

Detailed Comparison with Published Studies:

- Comparison with M. A. A. Al-Dulaimi et al. (2023): Al-Dulaimi et al. implemented an 8-tap FIR audio filter in VHDL on an Altera DE1 FPGA using an integrated WM8731 codec. While an 8-tap architecture optimizes resource usage and reduces processing latency, it suffers from a wide transition band. In contrast, this study implements a 32-tap filter using a Hamming window, achieving a significantly narrower transition band and enhanced stopband attenuation (-50 dB to -80 dB). Furthermore, while Al-Dulaimi et al. focused on hardware peripheral interfacing (I2C, S2P), this work emphasizes rigorous co-verification between floating-point MATLAB models and Verilog hardware simulations using quantitative error metrics ($RMSE = 0.0060$, $AND_{max} = 0.0095$).
- Comparison with S. Bhavani et al. (2023): Bhavani et al. synthesized a 16-tap low-pass FIR filter using MATLAB HDL Coder for automated code generation. While automated synthesis reduces development time, hand-crafted Verilog HDL (as used in this study) provides direct architectural control over micro-hardware structures—specifically managing the 8-bit delay line, configuring the 40-bit accumulator register (acc), and explicitly managing fixed-point scaling via 15-bit right-shifting. This structural control allows for precise evaluation of quantization noise and bit truncation effects.
- Comparison with Abdulbasit M. Sabawi: Sabawi designed an FIR audio filter in Quartus II and validated hardware functionality using an oscilloscope with frequency sweep inputs (100 Hz to 20 kHz). Sabawi's approach provides qualitative visualization of filter magnitude response across the audio spectrum. By contrast, this study utilizes a dual-tone test signal (500 Hz passband / 2500 Hz stopband) and exports 161 sample records (fir_input.hex) to perform sample-by-sample quantitative verification between ModelSim and MATLAB.

5.4. Limitations and Future Work

Limit:

Verification was conducted via simulation only; hardware synthesis and physical FPGA deployment were not performed, leaving logic resource utilization and maximum clock frequency unmeasured.

Testing was limited to a single dual-tone input signal (500 Hz + 2500 Hz) and a fixed set of design parameters.

Bit truncation was applied during shift operations without convergent rounding logic, introducing minor arithmetic bias.

The test dataset was limited to 161 discrete samples rather than continuous streaming audio or long-duration signals.

The direct-form architecture instantiates 32 parallel multipliers, leading to high hardware resource consumption.

Future Work:

Implement convergent rounding logic (adding 2^{14} prior to arithmetic right shifting) to reduce truncation error.

Expand input and output dynamic ranges to higher bit-widths for improved precision.

Exploit coefficient symmetry to fold the architecture, reducing multiplier utilization by half.

Evaluate alternative window functions (e.g., Hann, Blackman) and higher filter orders to analyze roll-off sharpness and stopband attenuation.

Synthesize and deploy the design onto an FPGA platform integrated with ADC/DAC codecs for real-time audio filtering.

Develop reconfigurable high-pass and band-pass modules alongside self-checking testbenches for automated PASS/FAIL reporting.

5.5. Conclusion

This project successfully executed the design, modeling, and hardware simulation of a low-pass FIR filter:

1. **MATLAB Design:** Synthesized a 32-tap low-pass FIR filter with Fs = 8000 Hz, Fc = 1000 Hz using a Hamming window, achieving over 50 dB of stopband attenuation.

2. **Simulink Verification:** Evaluated a dual-tone signal (500 Hz + 2500 Hz), demonstrating retention of the 500 Hz component (amplitude ≈ 0.69) and full attenuation of the 2500 Hz component.

3. **Verilog / ModelSim Simulation:** Constructed the fir32_lowpass module and testbench, processing 161 samples in 1626 ns and logging results to fir_output.txt while validating Q15 hardware coefficient alignment.

4. **Co-Verification:** Demonstrated strong alignment between MATLAB and ModelSim outputs, yielding a maximum absolute error of 0.0095 and an RMSE of 0.0060.

This study provided valuable insights into FIR filter theory, windowing techniques, Q15 fixed-point conversion, and hardware co-verification methodology comparing software algorithms to synthesizable HDL designs.

References

[1] A. V. Oppenheim and R. W. Schafer, Discrete-Time Signal Processing, 3rd ed. Pearson, 2010.
[2] M. A. A. Al-Dulaimi, H. A. Wahhab, and A. A. Amer, "Design and Implementation of Communication Digital FIR Filter for Audio Signals on the FPGA Platform," Journal of Communications, vol. 18, no. 2, pp. 89–96, Feb. 2023, doi:10.12720/jcm.18.2.89-96.
[3] R. A. Losada, Practical FIR Filter Design in MATLAB, Rev. 1.0, The MathWorks, Inc., Natick, MA, USA, Mar. 31, 2003.
[4] MathWorks, DSP System Toolbox Documentation
[5] MathWorks, “Filter Designer,” MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/signal/ref/filterdesigner-app.html. [Accessed: Nov. 1, 2026].
[6] Intel Corporation, “Using the ModelSim-Intel FPGA Simulator by Drawing Waveforms,” Intel FPGA University Program, Jan. 2020. [Online]. Available: https://community.intel.com/cipcp26785/attachments/cipcp26785/quartus-prime-software/60932/1/ModelSim_GUI_Introduction.pdf